

InBetween — Developer Onboarding & Architecture

A practical map of the whole system: what it does, the stack and *why* each tool is used, the repo layout, the data model, the core pipeline, and the deploy/ops story. Written for a developer joining the project cold.

File references point to directories/key files rather than line numbers (those drift). Search the symbol name to find the exact spot.

1. What the product does

InBetween is a **dance-coaching mobile app**. The core loop:

1. A **coach records a lesson** (audio) — private (1 student), couple (2 dancers), or group (many).
2. The audio is **transcribed** (AssemblyAI) and an **LLM (Claude) extracts "focus points"** — concrete things the student must work on, each tagged with a **tier** (**critical** / **important** / **supporting**) and a dance category (Latin / Ballroom).
3. The coach **reviews** the AI-generated focus points (or they auto-publish after 18h).
4. The **student trains** each focus point in short practice sessions; progress shows as **X / N** (**practice_count** / **train_target**).
5. The app computes a **readiness %** for the next lesson.
6. At the next lesson the coach does a **debrief** (keep / retire focus points), and the system **reconciles** old + new focus points down to a max of 3 per dance category.

Two main roles: **coach** and **student** (+ a small **trainer/admin** role gated by a specific email for QA/feedback).

2. Stack & tools (and why)

Frontend — React Native / Expo

Tool	Version	Why
Expo (SDK 54) + React Native 0.81 / React 19	~54	Managed RN with prebuild + EAS. Custom native modules where needed.
@react-navigation (native-stack + bottom-tabs)	^7	Routing. Not Expo Router. Role-based: separate Auth / Student / Coach / Trainer navigators, code-split & lazily loaded for fast cold start.
@supabase/supabase-js	^2.99	DB, Auth, Realtime, Storage — the single backend client.
@react-native-async-storage/async-storage	2.2	Local persistence: session, role cache, per-screen cold-start caches.
expo-audio + custom native modules	—	Lesson recording. See custom modules below.
expo-notifications	~0.32	Push (coach requests, transcript ready, focus updates, reconcile).
expo-updates	~29	OTA JS/asset updates (see Deploy).
react-native-svg, expo-linear-gradient, expo-haptics	—	Charts (RadarChart, MetricGauge), gradients, tactile feedback.
Fonts	—	Bricolage Grotesque (body) + TT Travels Next (titles/onboarding). ⚠️ <code>Fonts.ttfRegular/ttMedium</code> are <i>Bricolage</i> , not TT Travels — use the <code>travels*</code> aliases.

Custom native modules (modules/): - `continuous-audio-recorder` — records class audio **in the background / screen-off** (essential during a lesson). - `audio-route-picker` — pick the input (built-in mic, Bluetooth, external **DJI Mic**). - `live-activities` — iOS Dynamic Island / lock-screen "Recording in progress". - `local-recording-files` — indexes locally-stored recordings (e.g. DJI Mic WAV) before upload.

Backend — Supabase

Piece	Why
Postgres + RLS	Source of truth. RLS gives multi-tenant isolation (coach ↔ their students, couple ↔ partners). All RLS-bearing RPCs are SECURITY INVOKER so server-side reads enforce the exact same access as the client.
Edge Functions (Deno/TypeScript)	All AI/transcription orchestration + privileged writes. Run with the service-role key (bypass RLS) and do manual auth checks in code.
Storage (<code>class-audio</code> bucket)	Holds audio chunks: <code>class-audio/{user_id}/{recording_id}/{idx}.m4a</code> . Coach can only read/write their own <code>user_id/</code> folder.
pg_cron + pg_net	Scheduled sweeps (publish expired focus points, retry stuck transcriptions, monitoring report) that call edge functions over HTTP.
Server-side bundled RPCs	Perf: the free-tier connection pooler serializes round-trips, so N client queries become a multi-second "staircase". Several RPCs collapse a screen's queries into one call (see §8).

AI / transcription

Provider	Used for
AssemblyAI	Primary transcription (per-chunk, async via webhook).
Anthropic Claude	<code>yoda-extract</code> (focus-point extraction) → Claude Sonnet 4.6 ; <code>ai-chat</code> + transcript translation → Claude Haiku 4.5 ; <code>monitor-report</code> → Sonnet.
OpenAI Whisper	<code>whisper-transcribe</code> — fallback transcription.

All LLM calls are **proxied server-side** (keys never shipped to the client). Cost is logged per call to `ai_call_logs`. (AssemblyAI's key is the one exception historically embedded client-side — see §7.)

Build / infra

Tool	Why
EAS Build	iOS production builds (<code>--local</code> on CI). <code>appVersionSource: remote</code> + <code>autoIncrement</code> → EAS owns the build number.
GitHub Actions (<code>.github/workflows/testflight.yml</code>)	CI: build on a macOS runner + submit to TestFlight . Triggers: push to <code>main</code> or <code>workflow_dispatch</code> .
expo-updates / EAS Update	OTA updates on channel <code>production</code> , <code>runtimeVersion = appVersion</code> .
App Store Connect API (<code>.p8</code>)	Automated TestFlight upload via <code>xcrun altool</code> .
inbetween-admin (separate Next.js repo on Vercel)	QA gate: an admin approves a class (<code>class_inputs.admin_approved_at</code>) before its focus points can publish.

3. Repo layout

```
src/
  navigation/      AuthNavigator, StudentAppNavigator, CoachAppNavigator, TrainerNavigator
  screens/         student screens (HomeScreen=Train, LogScreen, ProfileScreen, FocusSessionScreen)
  coach/           coach screens (StartClassScreen, DashboardScreen, StudentDetailScreen,
                  ActionNeededScreen, FocusValidationScreen, CoupleDetailScreen...)
  components/      shared UI (TabBar, Skeletons, BottomSheet, charts) + coach/ sheets (Reconciler)
  context/         CoachDataContext (coach batched loader), ProfileContext (student)
  storage/         data-access layer: storage.js (student), coachStorage.js, coupleStorage.js,
                  activeCoachClass.js, recordingQueue.js, notificationsStorage.js, hydrate.js
  services/        supabase client, auth/role, notifications, upload worker, local-recording sync
  utils/           algorithm.js (focus ordering/slots), danceCategory.js, studentMetrics.js, normalize
  theme/           colors, fonts, spacing

supabase/
  migrations/      ~all schema + RPCs (timestamped .sql files)
  functions/       edge functions (Deno) – one folder per function
  _shared/         shared TS libs: yoda-score.ts, finalize-recording.ts, transcript.ts, normalize
  config.toml      per-function verify_jwt settings

docs/             specs (e.g. FOCUS_POINTS_REWORK_SPEC.md) + this file
modules/          custom native modules
App.js            root: resolve session+role → mount the right navigator
eas.json, app.json, .github/workflows/testflight.yml
```

App entry (App.js): cold-start-critical. Reads the session + role from a 2-tier cache (auth metadata → AsyncStorage), then conditionally mounts **one** navigator (Auth / Student / Coach / Trainer) so unused code isn't loaded.

State management: no Redux. React **Context** + AsyncStorage caches. - **CoachDataContext** — coach side, **two-wave** loader: wave 1 (students + user) unblocks the first screen; wave 2 (activity feed, notes, action counts) loads deferred. 60s in-memory TTL + refresh on app foreground / push received. - **ProfileContext** — minimal student profile (avatar/initials).

4. Data model (the important tables)

Identity & relationships

- **users** — extends Supabase Auth. **role** (**coach** / **student**), **dance_style**, **studio_id**, **latin_coach_id** / **ballroom_coach_id** (a student can have a coach **per style** — possibly the same coach for both), **couple_id**.
- **coach_requests** — student↔coach link, **per category** (**latin** / **ballroom** / null). Source of truth for "this coach coaches this student in this style".
- **couples**, **couple_requests**, **couple_coach_requests** — couple entity, pairing handshake, and per-style couple-coach designation.
- **studios** — shared studio entity.

Lessons & recording pipeline

- **class_recordings** — a recording session. **status** (**recording** → **ready** → **transcribing** → **completed** / **failed** / **discarded**), **expected_chunks**, **audio_folder**, **class_input_id**, **last_heartbeat_at**, **meta** (event log).
- **class_recording_chunks** — one row per ~3-min chunk. **idx**, **status** (**uploaded** → **transcribing** → **transcribed**), **storage_path**, **assemblyai_job_id**.
- **class_inputs** — the processed lesson. **lesson_type** (**private** / **couple** / **group** | **public**), **transcript**, **raw_ai_json** (Claude output), **class_summary**, **dance**, **status** (**scored** ...), **admin_approved_at** (QA gate), **created_at** (⚠️ **this is the canonical "class date"**, not the processing time).
- **class_input_students** / **attendance_responses** — group attendance (real attendance lives in **attendance_responses.attended**).

Focus points (the heart of the app)

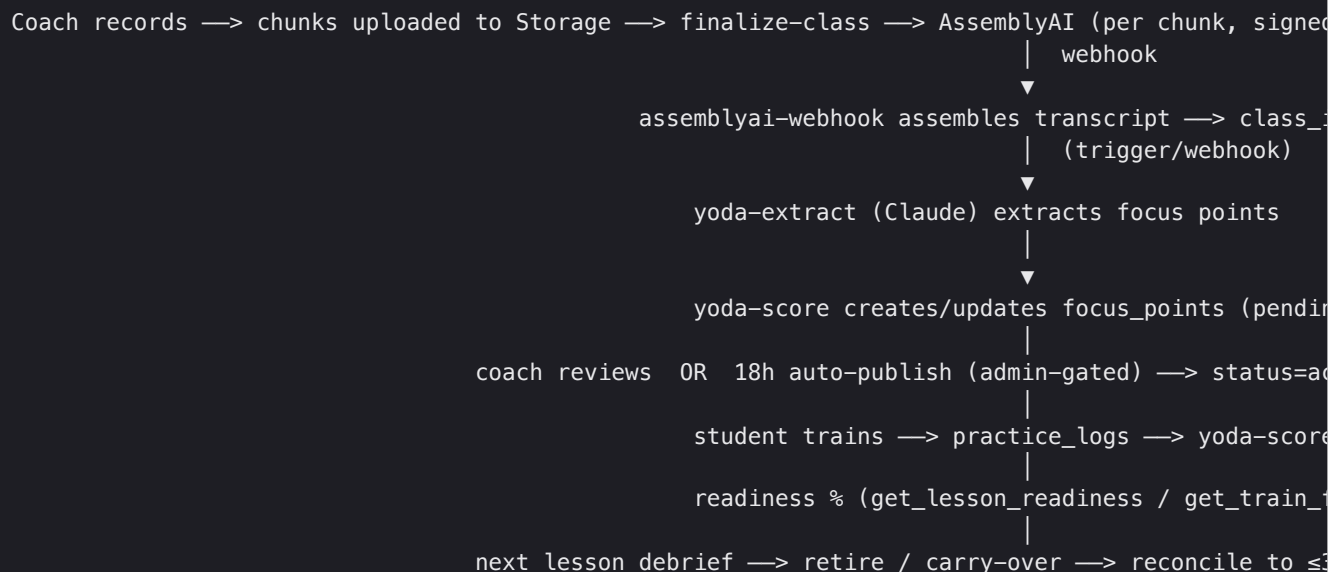
- **focus_points** (solo) and **couple_focus_points** (couple). Key columns:
- **tier** — **critical** / **important** / **supporting**. **Display rank only** + seeds the train target.
- **train_target** (int) — how many sessions to "train" it. Seeded from tier (**critical 3, else 2**), **+2 each time it's carried over**; never decreases.
- **practice_count** (int) — sessions done; **never reset**.
- **status** — **pending_coach** (awaiting review) → **active** (visible to student) → **past** (retired). (**past_candidate** is **dead/removed**.)
- **is_held** — marks a **"Not yet" carry-over** from a debrief.
- **dance[]** — for Latin/Ballroom categorization (untagged = both categories).
- **coach_review_deadline** — 18h auto-publish timer.
- **is_other**, **is_deleted**, **is_archived**, **group_fp**, **shared_group_id**, **class_input_id**, **source_class_input_id**.
- **base_score**, **coach_signal**, **lessons_since_mentioned** — **inert** (legacy scoring; pending "Niveau 2" column drop).

Progress shown to the student = **practice_count** / **train_target**. Readiness % = $\text{sum(done)} / \text{sum(target)}$ over the most-recent class's focus points. There is **no score/decay engine** anymore — a focus point lives until the coach validates it at a debrief or the student archives it.

Support tables

practice_logs / **couple_practice_logs** (training sessions; feeling + motivation), **merge_requests** (AI-proposed duplicate merges), **notifications**, **coach_messages** (student Q&A), **yoda_score_decisions** / **ai_*** tables (AI feedback/training), **ai_call_logs** (cost), **monitoring_reports**.

5. The core pipeline (end to end)



Step detail:

1. **Record** — `StartClassScreen` starts `continuous-audio-recorder`; chunks are uploaded to the `class-audio` Storage bucket and tracked in `class_recording_chunks`. A `class_recordings` row tracks the session (heartbeat every ~1s).
2. **Finalize** — on "Done", the client sets the recording `ready` + `expected_chunks` and calls `finalize-class` → `_shared/finalize-recording.ts`. It creates an **AssemblyAI job per chunk** using a **signed URL** from the `class-audio` bucket (the audio is *not* uploaded directly to AssemblyAI from the device).
3. **Transcribe** — AssemblyAI calls `assemblyai-webhook` (auth via `X-Webhook-Secret`) as each chunk completes; non-English is translated via Claude Haiku. When all chunks are done it **assembles the transcript** and creates the `class_inputs` row.
4. **Extract** — `class_inputs` INSERT triggers `yoda-extract` (Claude Sonnet 4.6): produces per-student focus points (`title`, `subtitle`, `context`, `drill`, `tier`, `category`, `dance[]`, `merge_action`, `coach_signal`) + group `shared_focus_points` + a `class_summary`. Strict style rules (e.g. no hyphens), retries on 5xx/429, cost logged.
5. **Score / persist** — `yoda-score` writes to `focus_points`: new ones as `pending_coach` (+18h deadline); re-mentions `auto_merge` (keep `practice_count`, `train_target` += 2); uncertain matches → `merge_request` + coach notification.
6. **Review** — coach approves in `ActionNeededScreen` / `FocusValidationScreen`, or the **18h auto-publish** fires (`publish-expired-fps` cron + a piggy-back in `yoda-score`) — but **only if the class is admin-approved** (`admin_approved_at`) and, for group focuses, the student confirmed attendance.
7. **Train** — student logs a session in `FocusSessionScreen` → `practice_logs` → `yoda-score` bumps `practice_count` (nothing else).
8. **Readiness** — Train screen (`HomeScreen`) and Profile read `get_train_focus` / `get_lesson_readiness` (per Latin/Ballroom toggle, persisted in AsyncStorage key `train_category_filter`).
9. **Debrief** (`StartClassScreen.finishDebrief`) at the next lesson — see §6.

6. Focus-point lifecycle: debrief + reconciliation (current model)

At a private debrief the coach reviews the previous class's focus points: - **Untouched** → **retired** (`status=past`). **Default-retire is intentional** — a focus point lives only until validated. - **"Not yet"** → **kept** (`is_held=true` , carried over; gets `+2` target when reconciled into the next class). Kept focuses **stay visible/trainable** in solo readiness/Train (the old "hide until 100% then auto-archive after 15 min" behavior was removed for solo). - **"Good"** → **retired** (`past` , explicit done). - The **Done** button is never gated.

Reconciliation — max 3 active focus points PER dance category (a 2-style dancer keeps up to 3 Latin and 3 Ballroom; they're never mixed): - Carry-over is `critical` / `important` → **silent auto-resolve**: drop the lowest new `supporting` (server-side, in `yoda-score`). - Carry-over is `supporting` → **coach picks** via `ReconcileFocusSheet` (notified with type `focus_reconcile_needed`). If not actioned, the **18h auto-publish** resolves it in favor of the carry-over (`autoResolveCarryover`). - The coach picks the **style** when starting a class with a 2-style student (a Latin/Ballroom popup), which scopes the readiness **and** the debrief default-retire to that style.

Solo vs Couple vs Group: - Private — 1 student, up to 3 focus points; full debrief + reconciliation. - **Group** (`group` / `public`) — many students, up to **2** shared focus points (one row per student, linked by `shared_group_id`); publishes only to students who confirmed attendance. - **Couple** — shared focuses live in `couple_focus_points` (keyed by `couple_id`). ⚠ Couple **reconciliation + held-visibility are a known follow-up** (the per-category reconcile + "Not yet stays visible" changes are currently **solo-only**).

7. Infrastructure & deployment

EAS / versions

- `eas.json` : `cli.appVersionSource = remote` , production profile `autoIncrement: true` , `channel: production` . **EAS owns the iOS build number** (the `"1"` in `app.json` is ignored). The **marketing version** comes from `app.json` .
- `app.json` : `runtimeVersion.policy = appVersion` . Bumping `version` (e.g. `1.7.0` → `1.7.1`) **forces a new native build** (an OTA only applies within the same runtime version).

CI (`testflight.yml`)

- Triggers: **push to main** or manual `workflow_dispatch` . (Both build from `main` .)
- macOS runner, Xcode 26 pinned, `eas build --platform ios --profile production --local` → `.ipa` → **TestFlight** via `xcrun altool` (ASC `.p8` key from GitHub secrets).
- ⚠ Don't trigger **cloud** EAS builds (credits). Use the local CI build or `eas build --local` .

OTA vs native build

- **OTA** (push to the update channel): JS/asset-only changes.
- **Native build** (TestFlight): native modules, permissions, plugin changes, or a `version` bump (runtime version change).

Deploy a backend change

- **Migrations:** apply the `.sql` (e.g. via the Supabase Management API query endpoint, or `supabase db push`). Mind ordering (a migration that uses a new column must run after the one that adds it).
- **Edge functions:** `supabase functions deploy <name> [...]`. A function and the `_shared` libs it imports are bundled at deploy — redeploy every function that imports a changed `_shared` file. Check `verify_jwt` in `config.toml` before deploying (most internal/webhook/cron functions are `verify_jwt=false`).

Secrets

- `.env` (local dev / CLI): `EXPO_PUBLIC_SUPABASE_URL`, `EXPO_PUBLIC_SUPABASE_ANON_KEY`, `EXPO_PUBLIC_ASSEMBLYAI_API_KEY`, `SUPABASE_PROJECT_REF`, `SUPABASE_ACCESS_TOKEN`, `SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_DB_URL`, `ASSEMBLYAI_WEBHOOK_SECRET`.
- **GitHub secrets:** `EXPO_TOKEN`, `ASC_API_KEY_P8`, `ASC_KEY_ID`, `ASC_ISSUER_ID`, + the `EXPO_PUBLIC_*` build vars.
- **Edge-function secrets** (Supabase dashboard, `Deno.env`): `SUPABASE_SERVICE_ROLE_KEY`, `ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `ASSEMBLYAI_API_KEY`, `ASSEMBLYAI_WEBHOOK_SECRET`.

8. Edge functions & key RPCs

Edge functions (`supabase/functions/`)

Function	Trigger	Role
<code>finalize-class</code>	client "Done" / retry cron	Create AssemblyAI jobs for a recording's chunks (idempotent).
<code>assemblyai-webhook</code>	AssemblyAI callback	Assemble transcript, create <code>class_inputs</code> .
<code>yoda-extract</code>	<code>class_inputs</code> INSERT	Claude extraction → focus points + summary.
<code>yoda-score</code>	<code>practice_log</code> / <code>class_input</code> event	Persist/merge focus points; 18h auto-publish; reconciliation.
<code>publish-expired-fps</code>	cron (5 min)	Publish <code>pending_coach</code> focuses past their 18h deadline (admin/attendance gated) + auto-resolve carry-overs.
<code>transcribe-class-retry</code>	cron (5 min)	Re-finalize stuck <code>ready</code> / <code>transcribing</code> recordings.
<code>practice-log</code>	client	Insert a practice log (verifies <code>student_id == auth.uid()</code>), then invoke <code>yoda-score</code> .
<code>ai-chat</code>	client	Student↔coach AI Q&A (Claude Haiku).
<code>whisper-transcribe</code>	client	Fallback transcription (OpenAI Whisper).
<code>get-teacher-context</code> , <code>attendance-response</code> , <code>student-class-score</code> , <code>monitor-report</code> , <code>telegram-webhook</code>	various	AI context, group attendance, trainer scoring, daily health report, ops bot.

Performance RPCs (SECURITY INVOKER, "bundle a screen into one call")

- `get_lesson_readiness(p_user, p_category)` → most-recent-private readiness `{ focuses[], percent, minutesRemaining }`.
- `get_couple_readiness(p_couple, p_category)` — couple equivalent.
- `get_students_readiness(p_users[], p_category)` — batch (coach roster / dashboard rings).
- `get_train_focus(p_user, p_category)` — Train screen: solo + couple slots + readiness + session counts in one call.
- `get_student_profile(p_user, p_category)` — student Profile bundle (~15 queries → 1).
- `get_coach_student_detail(p_student)` — coach's StudentDetail bundle.
- `get_all_focus_points(p_user, p_category)` — All-focus-points (solo/couple/group) bundle.

These exist because the free-tier pooler serializes round-trips; collapsing N queries into one RPC avoids multi-second loads. When adding a screen, prefer a bundled RPC over a client-side query waterfall.

9. Conventions, gotchas & known gaps

- **RLS is SECURITY INVOKER everywhere** — RPCs enforce the caller's access. Edge functions use the **service-role key (bypass RLS)** and must do **manual auth checks** in code (e.g. `practice-log` verifies `student_id == auth.uid()`; `finalize-class` checks JWT role / ownership).
- **18h auto-publish is admin-gated** — focus points only auto-publish once `class_inputs.admin_approved_at` is set (the separate `inbetween-admin` web app). Before approval, the 18h timer doesn't count.
- **Class date = `class_inputs.created_at`** (not the processing/scoring time).
- **Group attendance truth = `attendance_responses.attended`** (`class_input_students.attendance` stays `'pending'` forever — dead column).
- **Recording recovery:** if the app is killed mid-recording, a `class_recordings` row can get stuck in `status='recording'` with uploaded chunks but no `class_input`. The retry cron only sweeps `ready / transcribing`, so these are **orphaned** — recover by setting `status='ready'` + `expected_chunks` and calling `finalize-class`. (A sweep that auto-recovers dead "recording" rows with chunks is a worthwhile TODO.)
- **Legacy scoring is inert** — `base_score`, `coach_signal`, `lessons_since_mentioned`, the `focus_score_history` audit, and the trainer "score history" screens are slated for a **"Niveau 2"** removal; don't build on them.
- **.env ships the anon key + project ref + (locally) the service-role key** — service-role key is server-only; never bundle it into the app.
- **Cost:** every LLM/transcription call logs to `ai_call_logs` (model, tokens, USD). Summable per `class_input_id`.
- **Open follow-ups:** couple reconciliation + couple held-visibility (solo-only today); coach-side per-style readiness in StudentDetail + the start-class roster % still use `category=NULL` for 2-style students (the start-class debrief itself is correctly scoped via the style picker).

10. Quick start for a new dev

1. `npm install`, copy `.env` (ask for the values).
2. Run the app: `npx expo start --dev-client` (a dev build must be installed on the device — it uses custom native modules, so Expo Go won't fully work).
3. Test accounts: a coach + a couple of students exist for manual testing (ask the team).
4. Backend changes: edit `supabase/migrations/` or `supabase/functions/`, deploy with the Supabase CLI (see §7). Type-check edge functions with `deno check supabase/functions/<fn>/index.ts`.
5. Read `docs/FOCUS_POINTS_REWORK_SPEC.md` for the focus-point model in depth.
6. Ship: bump `app.json version` for a native release → merge to `main` → `testflight.yml` builds + submits to TestFlight. JS-only fixes can go out as an OTA update instead.